

FITIUCA DE EXAMEN - SISTEME DE OPERARE (VARIANTA DETALIATA)

1. PROCESE

Procesul este un program aflat in executie care detine propriul spatiu de memorie si propriile resurse.

- contine PCB: PID, stare, registre, pointer la page table
- are spatiu de adrese separat
- comunicarea se face prin IPC

fork(), exec(), wait()

- fork() creeaza un proces copil
- in copil returneaza 0
- in parinte returneaza PID copil
- exec() inlocuieste complet codul procesului
- wait() blocheaza parintele pana termina copilul

In demand paging, fork foloseste Copy-on-Write: memoria nu este copiata imediat. Copierea apare doar la scriere.

2. THREAD-URI

- thread = fir de executie in interiorul unui proces
- thread-urile partajeaza cod, date si heap
- fiecare thread are stiva si registre proprii

Proces vs Thread

- proces: memorie separata, cost mare
- thread: memorie partajata, cost mic
- thread-urile comunica direct

3. SINCRONIZARE

Sectione critica

- zona de cod ce trebuie executata exclusiv
- problema apare din cauza race condition

Mutex

- ofera exclusivitate
- lock() - intra in sectiune critica
- unlock() - paraseste sectiunea critica

Semafor

- variabila contor ≥ 0
- wait(P): scade valoarea
- signal(V): creste valoarea

Deadlock

Deadlock apare daca sunt indeplinite simultan urmatoarele conditii:

- mutual exclusion
- hold and wait
- no preemption
- circular wait

Metode de tratare: prevenire, evitare (Banker), detectare, ignorare.

4. CPU SCHEDULING

Round Robin

PASII ALGORITMULUI:

- procesele sunt ordonate dupa momentul de sosire
- toate sunt puse intr-o coada FIFO
- fiecare proces primeste CPU pentru durata cuantei
- daca nu termina, este pus la finalul cozii
- se repeta pana toate procesele termina

EXEMPLU:

- P1: start 0, CPU 5
- P2: start 1, CPU 3
- Cuanta = 2
- Executie: P1 -> P2 -> P1 -> P2 -> P1

SJF Preemptiv (SRTF)

PASII ALGORITMULUI:

- se alege procesul cu timpul RAMAS minim
- daca apare un proces mai scurt, preia CPU
- algoritm preemptiv

EXEMPLU:

- P1: start 0, CPU 8
- P2: start 1, CPU 4
- La t=1, P2 preia CPU

5. MEMORIE - PAGING

Adresare

- adresa logica este formata din $p \mid d$
- p = numar pagina
- d = offset

FORMULE:

- dimensiune pagina = 2^d
- numar pagini = 2^p
- memorie virtuala = numar pagini * dimensiune pagina

Address Translation

PASII:

- CPU genereaza adresa logica
- se separa p si d
- page table traduce p in frame
- adresa fizica = frame * dim_pagina + d

Page Fault

PASII:

- pagina nu este in RAM
- se genereaza trap catre OS
- pagina este adusa din disk
- page table este actualizat
- instructiunea este reluata

LRU - Page Replacement

PASII:

- se parcurge sirul de pagini
- daca pagina nu este in frame apare page fault
- daca frame-urile sunt pline se elimina pagina cel mai demult folosita

EXEMPLU:

- frame-uri = 3

- sir pagini: 1 2 3 2 4
- fault-uri la 1, 2, 3, 4

6. FILE SYSTEMS

Indexare pe un nivel

PASII:

- $\text{pointer_size} = \text{word_size}$
- $\text{numar_pointeri} = \text{block_size} / \text{pointer_size}$
- $\text{dimensiune_maxima_fisier} = \text{pointeri} * \text{block_size}$

EXEMPLU:

- $\text{block_size} = 128 \text{ bytes}$
- $\text{pointer_size} = 2 \text{ bytes}$
- $\text{pointeri} = 64$
- $\text{dimensiune_maxima_fisier} = 64 * 128$

FAT

- fiecare intrare indica urmatorul bloc
- fisierele sunt lanturi de blocuri
- accesul secvential este rapid

7. DISK SCHEDULING

FCFS

- cererile sunt servite in ordinea sosirii
- $\text{distanța totală} = \text{suma diferențelor absolute}$

SCAN

- capul de citire merge intr-o directie
- serveste cererile intalnite
- la capat isi schimba directia

EXEMPLU:

- pozitie initiala = 50
- cereri = 10, 20, 70
- SCAN: 50 -> 70 -> 20 -> 10